Git / GitHub ⭐ ⭐ ⭐ — 3–4 YOE Scenario-Based Q&A PART-1

1. Developer accidentally pushed code directly to the main branch. What will you do?
First, I check what changes were pushed and whether they are already affecting the Production pipeline.
If the commit is incorrect, I prefer using git revert for a shared branch because it creates a new commit that safely reverses the previous change.
Wrong Commit
↓
git revert
↓
New Revert Commit
↓
Push
I would avoid git reset --hard on a shared branch because rewriting shared history can affect other developers.
Simple ga gurthupettuko:
Shared Branch → **Revert preferred**
Local/private work → **Reset can be considered**
Eng: If incorrect code is pushed to a shared branch, I would normally use git revert to safely undo the change without rewriting shared history.

---

2. Two developers modified the same file and there is a merge conflict. How will you resolve it?
First, I pull the latest target branch and identify the conflicting files.
Git marks the conflicting sections inside the file.
<<<<<<< HEAD
My Changes
=======
Other Developer Changes
>>>>>>> branch
I discuss with the developer if necessary, decide which logic should remain, manually resolve the conflict, and then:
git add <file>
git commit
Then I push the resolved changes and verify the pipeline.
Simple ga gurthupettuko:
Conflict → **Understand Both Changes → Resolve → Test → Commit → Push**
Eng: I resolve merge conflicts by reviewing both changes, selecting the correct logic, testing the merged code, and committing the resolved result.

---

3. A developer says, "My branch is working, but after merging into main the application is failing." What will you check?
I compare the branch with the latest main branch.
Possible reasons include:

- Merge conflict resolution
- Different configuration
- Dependency changes
- Environment variables
- Code from another developer
- Integration issues

Feature Branch
↓
Merge
↓
Main
↓
Failure
↓
Compare Changes
I also check the CI pipeline logs and identify exactly which commit introduced the failure.
Simple ga gurthupettuko:
Works Branch → Fails Main = **Check Integration Difference**
Eng: I would compare the feature branch with the target branch, inspect the merged changes and CI logs, and identify whether the failure was caused by integration or environment differences.

---

4. How do you find which commit introduced a Production issue?
I first identify the time when the issue started and correlate it with recent deployments and commits.
Useful Git commands include:
git log
git show <commit>
git diff
For a specific regression, git bisect can help identify the commit that introduced the problem by performing a binary search through commit history.

Known Good
↓
Git Bisect
↓
Test
↓
Good / Bad
↓
Problem Commit

Simple ga gurthupettuko:
Production Issue → **Recent Commits → Logs → Bisect if needed**
Eng: I would correlate the incident with recent commits and use Git history or git bisect when necessary to identify the commit that introduced the regression.

---

5. What is the difference between git revert and git reset?
git revert creates a **new commit** that reverses an earlier commit.
git reset moves the branch pointer and can rewrite local history depending on the reset mode.
Revert:
A → B → C → Revert C

Reset:
A → B → C
↑
Move Back

For shared branches, I generally prefer git revert.
Simple ga gurthupettuko:
Revert = **History safe ga undo**
Reset = **History pointer move**
Eng: git revert creates a new commit that undoes previous changes, while git reset moves the branch reference and can rewrite history, so reset should be used carefully on shared branches.

---

6. Developer accidentally committed a password to Git. What will you do?
First, I treat the credential as compromised.
I would:

1. Rotate/revoke the exposed credential.
2. Remove the secret from the code.
3. Check where the secret was pushed or exposed.
4. Clean the repository history if required.
5. Update the application to retrieve the secret securely.

For example:
Bad:
Git → password hardcoded

Good:
Git → Secret Reference
↓
Secrets Manager

Removing the secret from the latest file alone is not enough if it remains in Git history.
Simple ga gurthupettuko:
Secret Leak → **Rotate First → Remove → Clean History → Secure Storage**
Eng: I would immediately rotate the exposed credential, remove it from the code and history where appropriate, assess exposure, and move secret management to a secure secrets-management solution.

---

7. Git repository has become very large. How will you troubleshoot?
First, I identify what is consuming the repository size.
Common causes include:

- Large binaries
- Build artifacts
- Log files
- Generated files
- Old large files in Git history

I check repository history and determine whether Git LFS or external artifact storage is more appropriate.
Source Code → Git
Artifacts → Nexus / Artifact Repository
Images → Container Registry
Simple ga gurthupettuko:
Git = **Source Code**
Artifact Repository = **Build Artifacts**
Eng: I would identify large files and historical objects, remove inappropriate artifacts from source control, and use Git LFS or an artifact repository where appropriate.

8. GitHub webhook is configured, but Jenkins pipeline is not triggering. How will you troubleshoot?
I check the complete flow:
GitHub
↓
Webhook
↓
Jenkins
↓
Job
↓
Pipeline
I verify:

- Webhook delivery status
- Jenkins URL accessibility
- Webhook configuration
- Repository/job configuration
- Branch filters
- Jenkins credentials
- Jenkins logs

If GitHub shows successful webhook delivery but Jenkins does not trigger the job, I focus on Jenkins job configuration and trigger conditions.
Simple ga gurthupettuko:
Webhook issue → **GitHub Delivery → Jenkins Trigger → Job Config**
Eng: I would verify webhook delivery first, then check Jenkins trigger configuration, repository settings, branch filters, credentials, and Jenkins logs.

---

9. How do you protect the main branch?
I use branch protection rules.
Typical controls include:

- Pull request required
- Code review required
- Status checks required
- CI pipeline must pass
- Direct push restricted
- Force push restricted

Developer
↓
Feature Branch
↓
Pull Request
↓
Review
↓
CI Checks
↓
Merge
↓
Main
Simple ga gurthupettuko:
Main Branch = **No Direct Push**
Eng: I protect the main branch by requiring pull requests, reviews, successful CI checks, and restricting direct or force pushes.

---

10. Production deployment was successful, but you need to know exactly which Git commit is running. How will you identify it?
I make the application build immutable and attach the Git commit SHA or release version to the artifact.
Example:
Git Commit
↓
Build
↓
Artifact: app-8f32a1
↓
Deployment
The deployed application should expose or log its version where appropriate.
For Docker:
app:8f32a1
instead of relying only on a mutable tag like:

app:latest
Simple ga gurthupettuko:
Git SHA → **Artifact → Deployment → Traceability**
Eng: I use a unique Git commit SHA or release version in the build artifact so I can trace exactly which source revision is running in Production.

---

11. Why is using latest **Docker tag risky in Production?**
latest is a mutable tag. The same tag can point to different image versions over time.
For example:
Today:
latest → v1.0

Later:
latest → v1.1
This makes deployments and rollbacks less predictable.
I prefer immutable versioning:
app:1.0.0
app:1.0.1
app:<git-sha>
Simple ga gurthupettuko:
Production → **Versioned/Immutable Image Tags**
Eng: I avoid relying on mutable tags such as latest in Production and prefer immutable version tags or Git commit-based image tags for traceability and reliable rollback.

---

12. A developer wants to merge a feature, but CI is failing. Should you merge it?
Normally, no.
I first identify why CI is failing.
Pull Request
↓
CI
↓
❌ Failed
↓
Fix
↓
CI
↓
✅ Passed
↓
Merge
If the failure is unrelated infrastructure noise, I investigate and document it rather than blindly bypassing the check.
Simple ga gurthupettuko:
CI Fail → **Don't Blindly Merge**
Eng: I would normally block the merge until the CI failure is understood and resolved, unless there is an approved process for handling an unrelated infrastructure failure.

---

13. How do you handle a hotfix for a Production issue?
I create a controlled hotfix branch from the appropriate Production/release baseline.
Production
↓
Hotfix Branch
↓
Fix
↓
Test
↓
CI
↓
Production
After deploying the fix, I make sure the same change is also merged back into the main development branch so the fix is not lost in future releases.
Simple ga gurthupettuko:
Production Fix → **Hotfix → Deploy → Merge Back**
Eng: For a Production hotfix, I create a controlled branch from the release baseline, test and deploy the fix, and then merge the change back into the main development line.

---

14. Developer accidentally deleted a branch. Can you recover it?
Often, yes, if the relevant commit still exists in Git's references or reflog.
I first identify the last known commit SHA.
git reflog
Then I can recreate the branch from the appropriate commit:

git checkout -b recovery-branch <commit-sha>
The exact recovery approach depends on whether the branch was local, remote, and whether the commits are still reachable.
Simple ga gurthupettuko:
Deleted Branch → **Find Commit → Recreate Branch**
Eng: A deleted branch can often be recovered if its commits are still reachable, and I would use Git history or reflog to identify the last known commit and recreate the branch.

---

15. Explain a real-time Git → CI/CD flow.
Developer feature branch lo code change chestadu.
Pull Request create chestadu. CI pipeline automatically build, test, and code-quality checks perform chestundi.
Checks successful ayithe approved PR main branch ki merge avutundi.
Main branch merge ayyaka pipeline Docker image build chesi versioned image registry ki push chestundi. Deployment process aa exact image version ni Kubernetes/EC2 environment ki deploy chestundi.
Developer
↓
Feature Branch
↓
Pull Request
↓
Code Review
↓
CI Build
↓
Unit Tests
↓
SonarQube
↓
Docker Build
↓
Registry
↓
Deployment
↓
Kubernetes / EC2
↓
Monitoring
Simple ga gurthupettuko:
Branch → PR → Review → CI → Image → Registry → Deploy → Monitor
Eng: In a typical DevOps workflow, developers work on feature branches, create pull requests, and rely on automated CI checks before merging. After the merge, the pipeline builds and validates a versioned artifact or container image, pushes it to a registry, deploys the exact version to the target environment, and monitors the release.